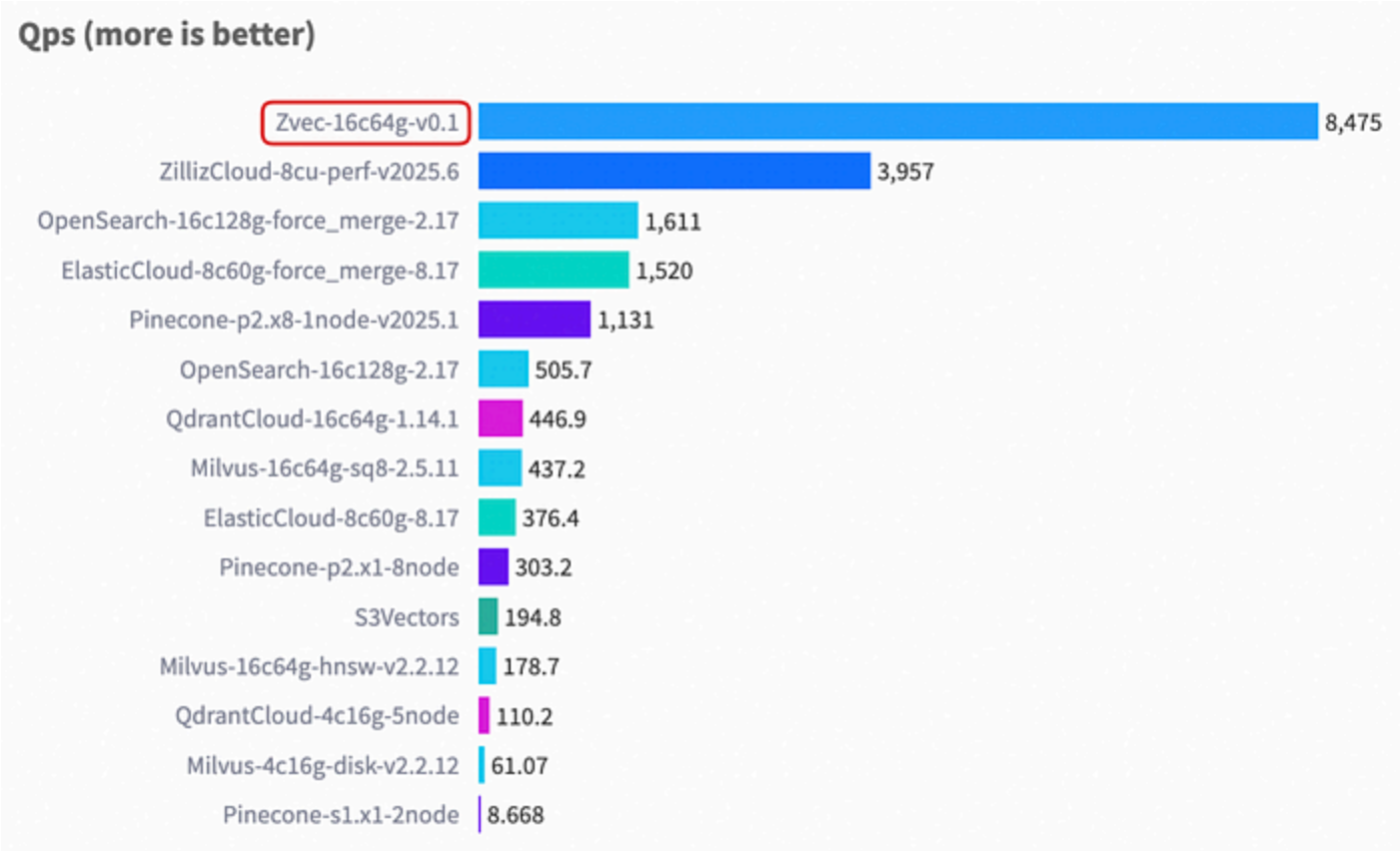
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Fastest Memory Layer for Agents: zvec vs. Chroma, Qdrant and Weaviate

Lightning-fast in-process vector database that is designed to embed directly into agentic AI applications.

Agent Native Follow
~11 min read · March 3, 2026 (Updated: March 3, 2026) · Free: No

Connection to your vector database is not free.

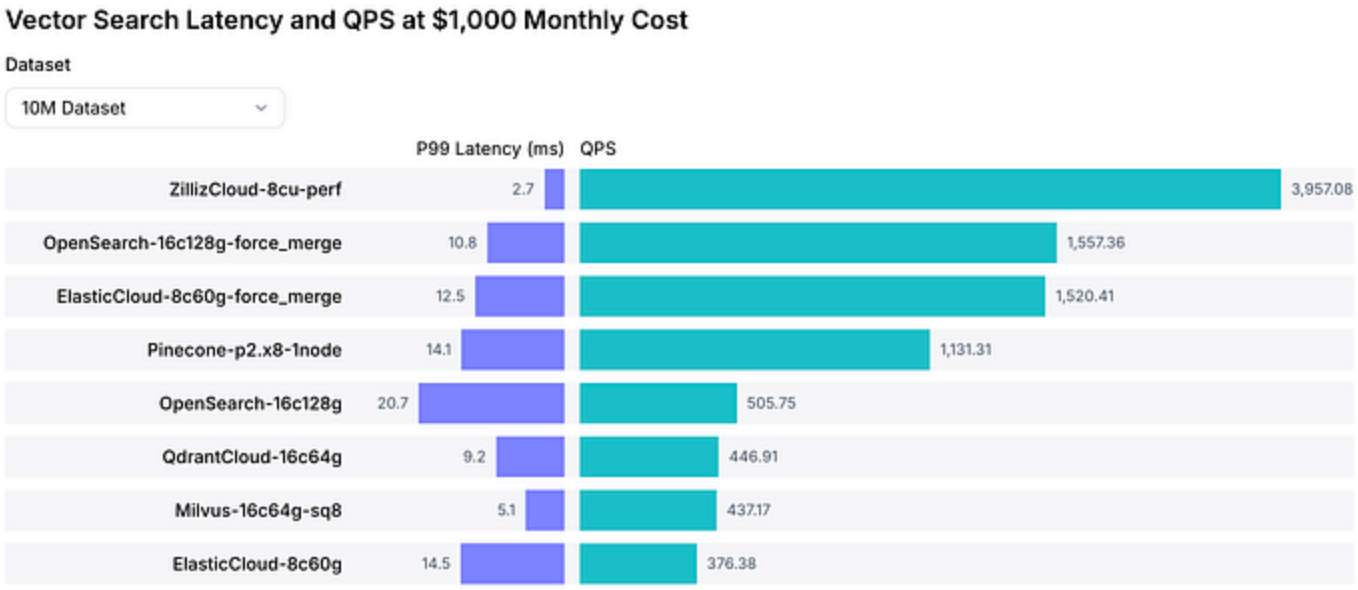
There is the network hop, the serialization overhead, the server you have to keep alive, the Docker container you inevitably forget to resource-limit.

I have watched production RAG pipelines where the retrieval leg (not the LLM inference, the retrieval) was the latency bottleneck.

An entire GPU cluster waiting on a TCP round-trip to a vector store running on a different node.

Alibaba just open-sourced something that directly addresses this problem.

It runs entirely inside your application process, and on the public VectorDBBench leaderboard it more than doubles the QPS of the previous number one ZillizCloud on the Cohere 10M dataset while reducing index build time. (more on this later)



And the part that caught my attention was a benchmark that compared zvec against FAISS, ChromaDB, and NumPy on a healthcare use case, where **zvec clears filtered queries at 0.5ms versus ChromaDB's 10ms+.**

Table 1 — Index Build Time

Library	Time	Notes
NumPy	~0 ms	Just holds a reference to the array
FAISS HNSW	0.03s	C++ batch HNSW construction
zvec HNSW	0.37s	C++ SIMD + persistent to disk
ChromaDB embed	1.21s	Python + hnswlib, with metadata index

Table 2 — Pure Vector Search (no filter, top-5, 50 queries)

Method	Avg Latency	QPS	Filter	Persist	Backend
FAISS HNSW	0.05 ms	18216	✗	✗	C++ Meta/SIMD
NumPy	0.20 ms	5027	✗	N/A	Vectorized numpy
zvec HNSW	0.50 ms	2012	✓	✓ disk	C++ Proxima/SIMD
ChromaDB embed	0.64 ms	1572	✓	opt.	hnswlib/Python

Table 3 — Filtered Search (age 40-70, top-5, 30 queries)

Method	Avg Latency	Filtering Approach
zvec + filter	0.82 ms	InvertIndex + HNSW graph (native C++)
ChromaDB + filter	15.86 ms	HNSW + hnswlib metadata post-filter
FAISS	N/A	No built-in filter; manual post-filter needed
NumPy	N/A	Brute-force then Python list filter

Practitioner benchmark

Later in this article, I will walk through the architecture decisions that make this possible, the resource governance model that makes it viable on edge devices, and the specific tradeoffs you should understand before adopting it.

The Tension Nobody Talks About

The retrieval path is often slower than the model inference it is supposed to support. When your GPU pipeline finishes in 50ms but your vector lookup takes 80ms because it crosses a network boundary, serializes the query, waits for the index node to run HNSW, deserializes the results, and ships them back, you have built a system where the **augmented** part is the bottleneck, not the **generation** part.

Anyone who has operated a production RAG system with a service-based vector store knows the pattern. The p50 looks fine. The p99 does not, because **TCP connections hiccup, garbage collection pauses hit the vector service, or the load balancer makes a suboptimal choice**. You end up over-provisioning the vector tier just to keep tail latency in check.

The standard options have well-known limitations:

- **Index libraries like FAISS**: Blazing fast at raw ANN search, but they are not databases. Kill the process and the index is gone. You end up building your own storage layer around them.
- **Embedded extensions like DuckDB-VSS**: Better, but constrained index choices, no quantization, and limited runtime resource control.
- **Service-based systems like Milvus or Pinecone**: Full-featured, but they require network calls, separate deployment, and operational overhead that is overkill for desktop tools, mobile apps, CLI utilities, or edge devices.

There is a gap in the middle: something that has database semantics (persistence, CRUD, crash recovery, hybrid queries) but runs inside your process like a library. That gap is exactly what zvec is designed to fill.

> Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **Bookmark it here:** Agentic SaaS Patterns Winning in 2026, packed with real-world examples, architectures, and workflows you won't find anywhere else.

What Is zvec, Exactly?

zvec is an open-source, embedded, in-process vector database released by Alibaba's Tongyi Lab under the Apache 2.0 license. It's essentialy **SQLite, but for vectors.**

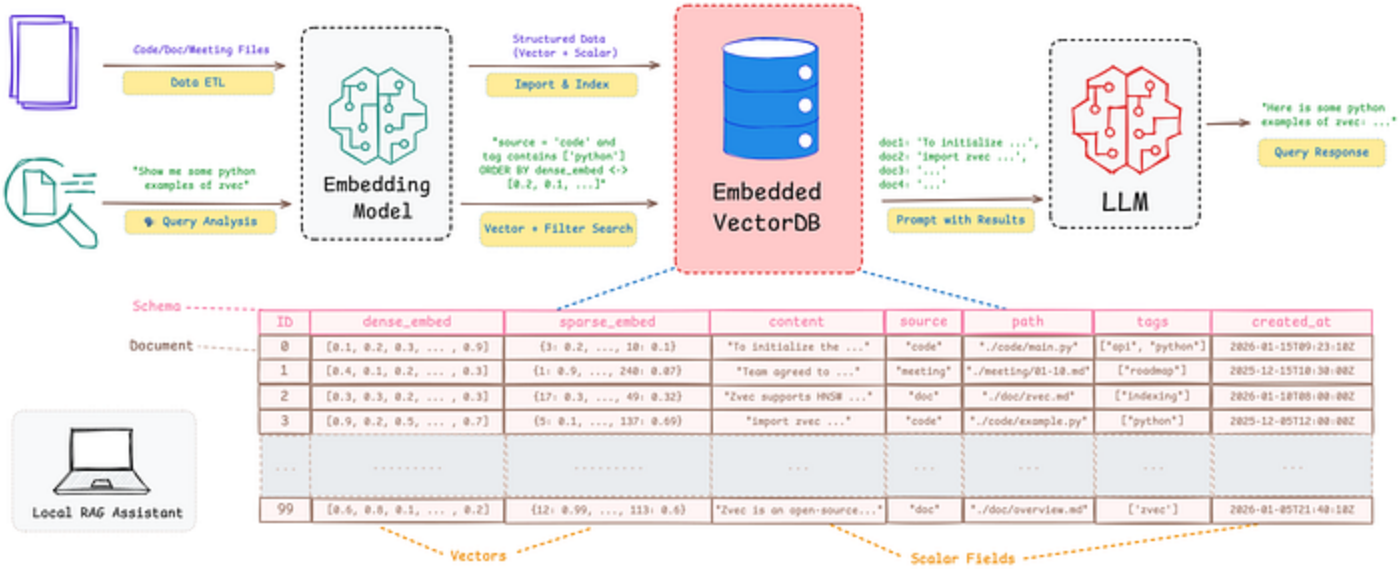
It is built on **Proxima**, Alibaba Group's production-grade vector search engine, the same engine that powers their own billion-scale internal systems for search, recommendation, and advertising. zvec wraps Proxima with a simpler Python API and an embedded runtime.

zvec at a Glance

- **Deployment model**: In-process library, no external service.
- **Engine**: Proxima, Alibaba's battle-tested production vector engine.
- **Language**: C++ core (81.3%), Python SDK. SWIG bindings.
- **Platform support**: Linux x86_64, Linux ARM64, macOS ARM64. Python 3.10–3.12.
- **Index type**: HNSW for vectors, inverted indexes for scalar filtering.
- **License**: Apache 2.0. 8.1k GitHub stars, 453 forks as of March 2026.

SQLite runs in your process, persists to a file, needs no configuration, and is the most deployed database engine in the world precisely because of that simplicity.

zvec makes the same bet for vector workloads because most applications need vector search plus metadata filtering, not a distributed cluster.



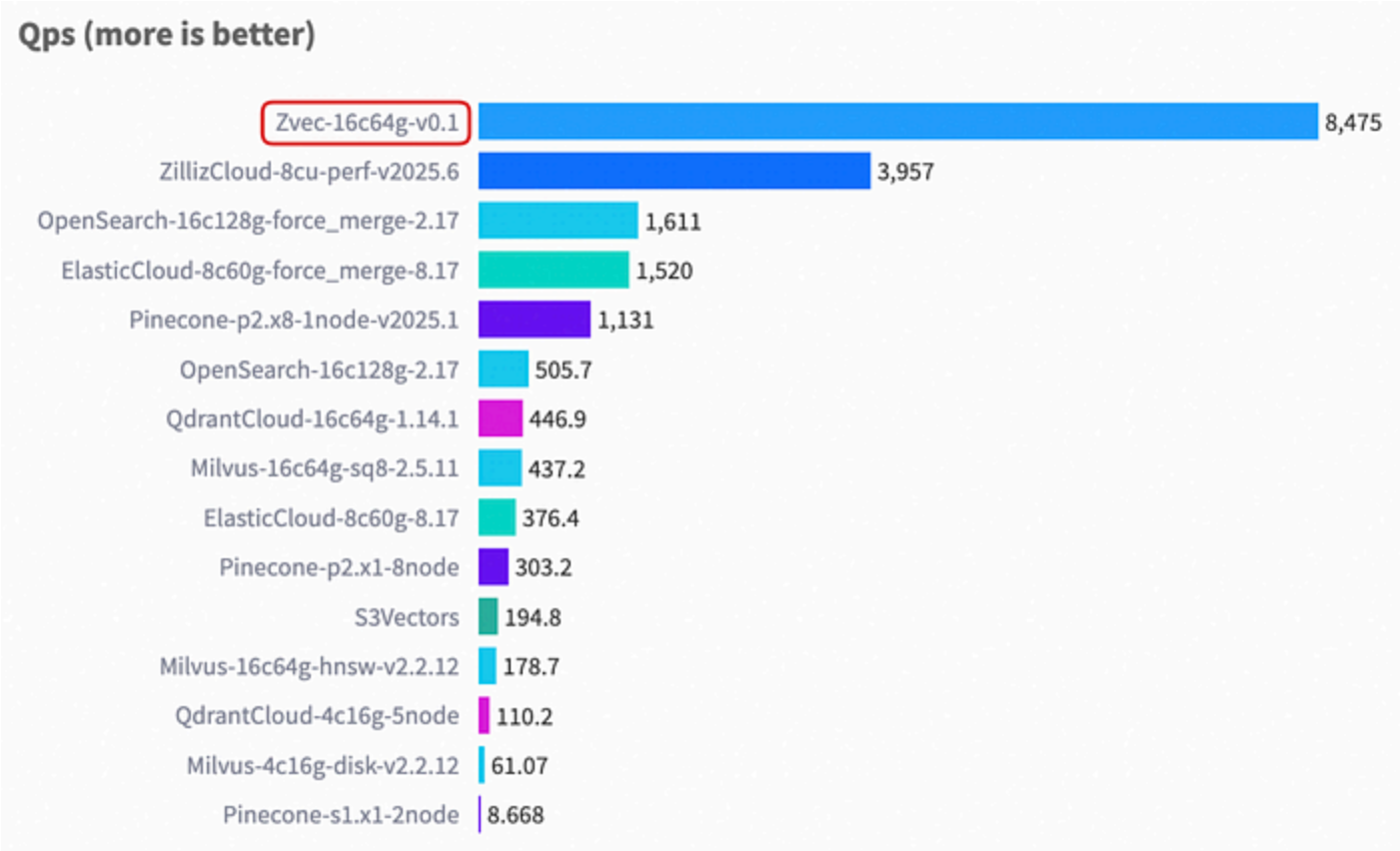
Where zvec Wins (and Where It Doesn't)

Let me start with the official numbers before getting to the independent ones, because together they tell a more complete story.

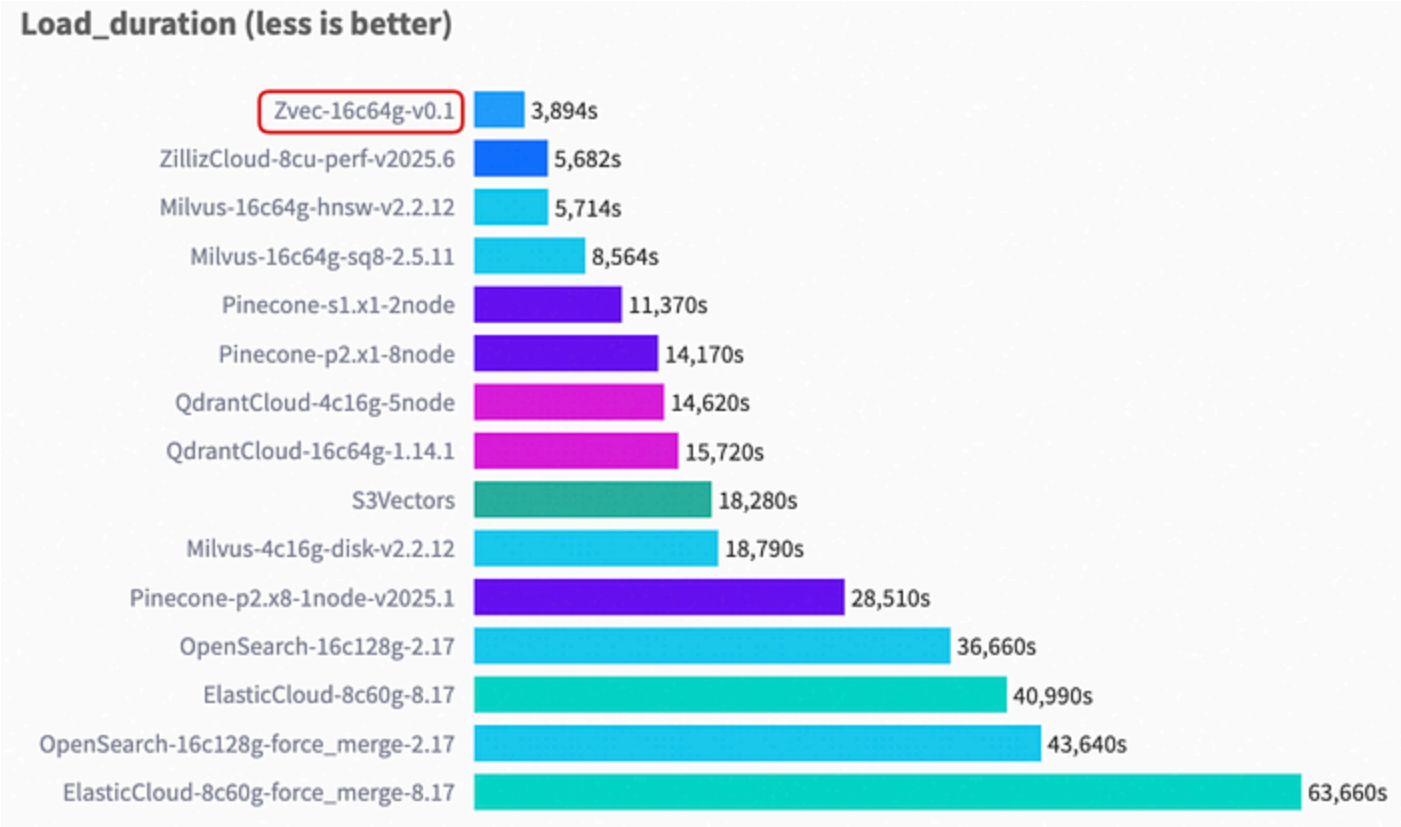
Official VectorDBBench results

zvec was benchmarked using VectorDBBench, an open-source framework widely adopted in the vector database community, on standardized datasets:

On **Cohere 10M** (10 million 768-dimensional vectors): zvec achieved **8,500+ QPS** with sub-millisecond latency, more than 2x the previous leaderboard leader (ZillizCloud) on comparable hardware and matched recall.

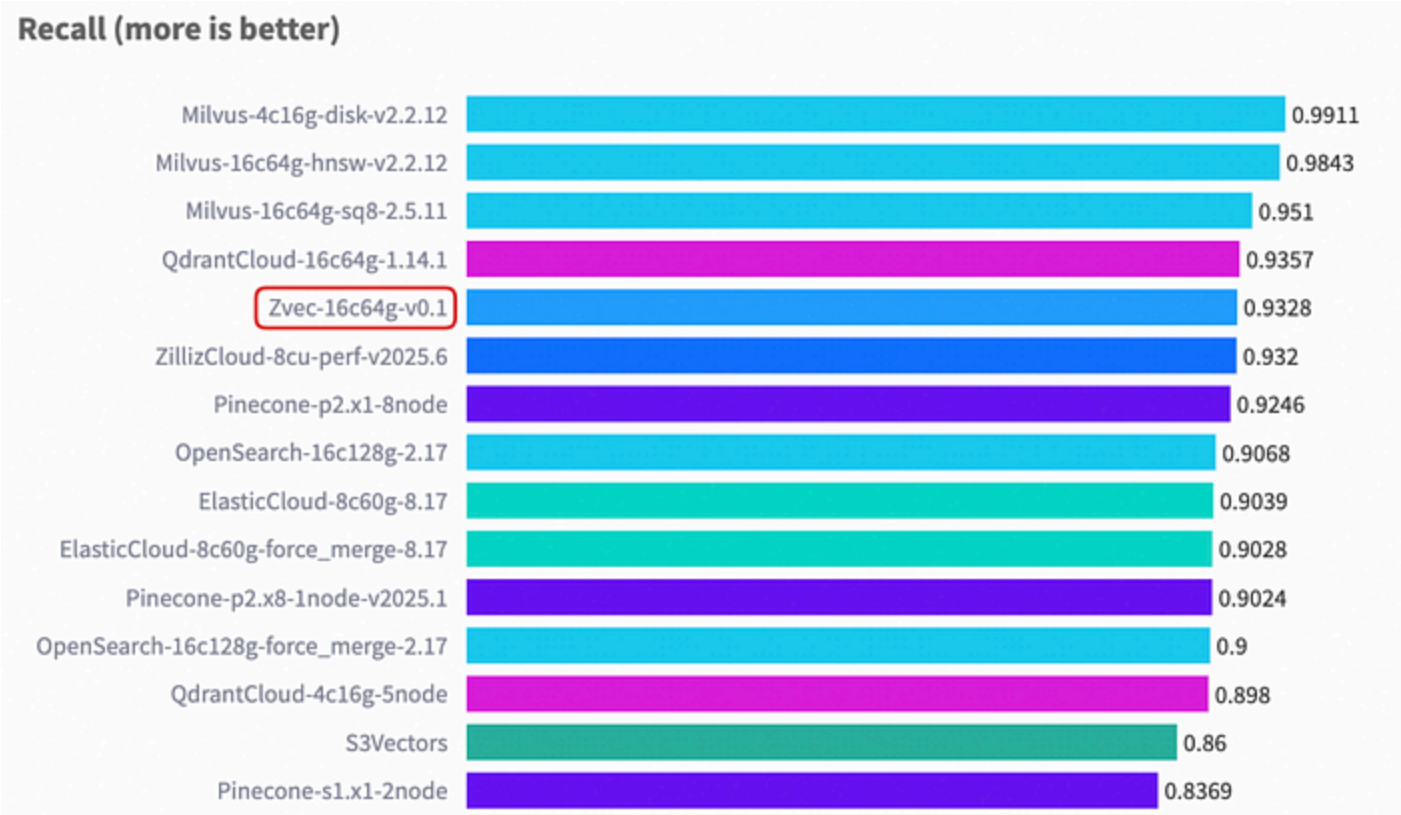


Index build time was approximately 1 hour for 10 million vectors on a g9i.4xlarge instance (16 vCPU, 64 GiB RAM), which also represents a substantial improvement over competitors.



SIMD-optimized distance computation, cache-friendly memory layouts, multi-threaded execution, and CPU prefetching.

The performance comes from the Proxima engine, not merely from being in-process.



Independent benchmarks: the nuanced picture

This is where it gets more interesting. A developer building semantic patient search for healthcare ran a 4-way benchmark with FAISS, zvec, ChromaDB, NumPy on 10,000 patient records with clinical embeddings.

The results are honest and worth internalizing:

Table 1 — Index Build Time

Library	Time	Notes
NumPy	~0 ms	Just holds a reference to the array
FAISS HNSW	0.03s	C++ batch HNSW construction
zvec HNSW	0.37s	C++ SIMD + persistent to disk
ChromaDB embed	1.21s	Python + hnswlib, with metadata index

Table 2 — Pure Vector Search (no filter, top-5, 50 queries)

Method	Avg Latency	QPS	Filter	Persist	Backend
FAISS HNSW	0.05 ms	18216	✗	✗	C++ Meta/SIMD
NumPy	0.20 ms	5027	✗	N/A	Vectorized numpy
zvec HNSW	0.50 ms	2012	✓	✓ disk	C++ Proxima/SIMD
ChromaDB embed	0.64 ms	1572	✓	opt.	hnswlib/Python

Table 3 — Filtered Search (age 40-70, top-5, 30 queries)

Method	Avg Latency	Filtering Approach
zvec + filter	0.82 ms	InvertIndex + HNSW graph (native C++)
ChromaDB + filter	15.86 ms	HNSW + hnswlib metadata post-filter
FAISS	N/A	No built-in filter; manual post-filter needed
NumPy	N/A	Brute-force then Python list filter

Practitioner benchmark

The key insight from this independent benchmark is that **the comparison that actually matters is zvec vs. ChromaDB, which has same feature tier.**

zvec wins clearly, especially on filtered queries (0.5ms vs 10ms+).

Comparing zvec to FAISS is apples-to-oranges because FAISS is an index library.

And the NumPy result is a useful calibration as HNSW overhead only pays off at scale. At 10K vectors, brute force wins. At 1M records, the HNSW-based engine is projected to be 38x faster.

Architecture: Why In-Process Matters More Than You Think

The architectural shift zvec represents is data locality.

When your vector engine runs inside your application process, several things become true simultaneously:

- **No RPC layer** means no serialization/deserialization overhead.
- **No network round-trip** so latency is deterministic and bounded by CPU, not by network jitter.
- **Direct memory access** for application to read vectors from the same address space.
- **Simpler failure domain** as your application process is the only thing that can fail where there is no separate service to monitor, restart, or scale.

As Maxime Grenu also noted (contributor to the project):

> When your GPU pipeline is fast, the CPU-side retrieval path becomes the bottleneck. An embedded vector engine is one of the most effective ways to eliminate that bottleneck.

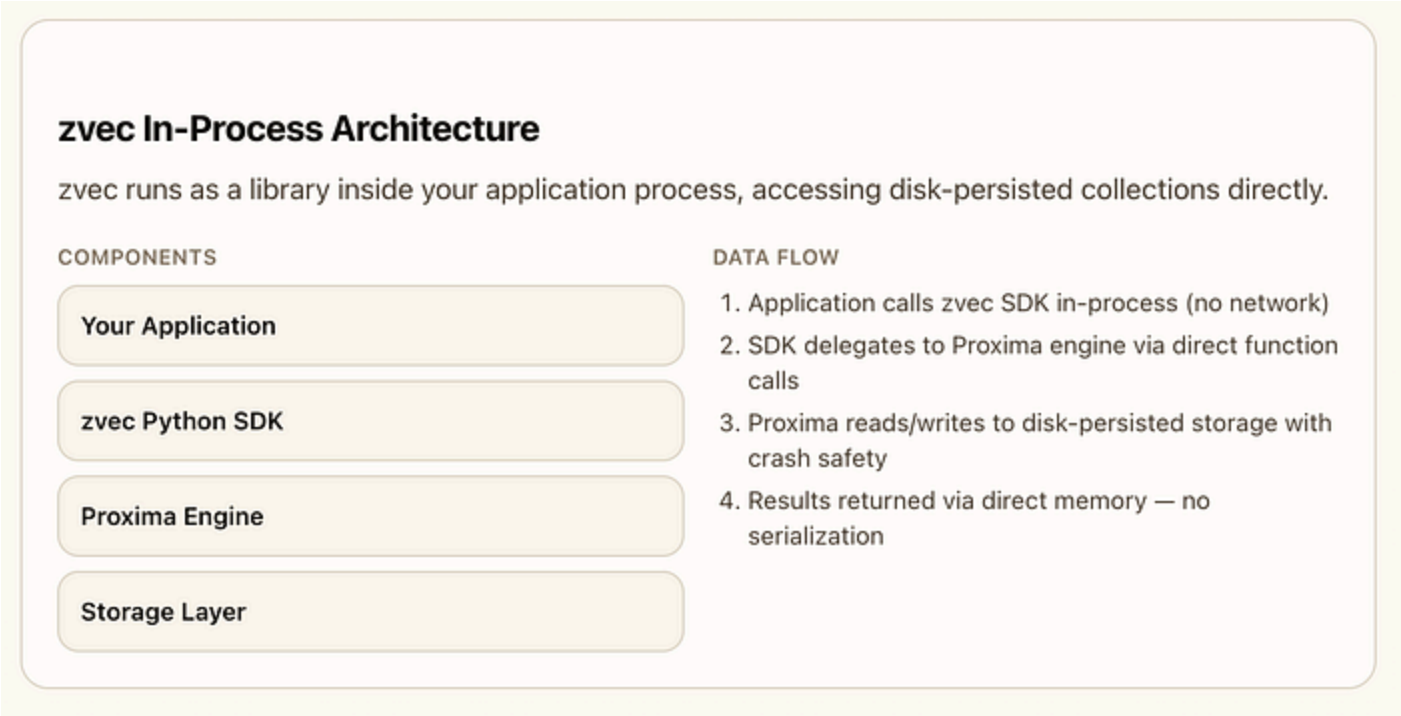
Here's a one minute example:

```python
import zvec

# Define collection schema
schema = zvec.CollectionSchema(
    name="example",
    vectors=zvec.VectorSchema("embedding", zvec.DataType.VECTOR_FP32, 4),
)

# Create collection
collection = zvec.create_and_open(path="./zvec_example", schema=schema,)

# Insert documents
collection.insert([
    zvec.Doc(id="doc_1", vectors={"embedding": [0.1, 0.2, 0.3, 0.4]}),
    zvec.Doc(id="doc_2", vectors={"embedding": [0.2, 0.3, 0.4, 0.1]}),
])

# Search by vector similarity
results = collection.query(
    zvec.VectorQuery("embedding", vector=[0.4, 0.3, 0.3, 0.1]),
    topk=10
)

# Results: list of {'id': str, 'score': float, ...}, sorted by relevance
print(results)
```

This is the same insight that made SQLite the most deployed database in the world. Most applications do not need a client-server database. They need a reliable, fast, local data store. The vector world is arriving at the same conclusion.

zvec In-Process Architecture

zvec runs as a library inside your application process, accessing disk-persisted collections directly.



RAG-Ready, Not Just ANN Search

Here is where zvec differentiates from pure index libraries.

A production RAG pipeline needs more than "find me the k nearest neighbors."

It needs:

- **Dynamic knowledge management.** Your knowledge base is not static. Documents are added, updated, deleted. Meeting notes from today replace yesterday's draft. Full CRUD is not optional — it is the minimum. zvec supports full create, read, update, and delete operations with schema evolution, so you can adjust index strategies as your metadata and query patterns change.
- **Multi-vector retrieval and fusion.** Modern RAG often combines multiple embedding channels — a dense semantic embedding plus a sparse keyword embedding, for example. zvec supports multi-vector joint queries natively. It also ships a built-in reranker that supports both weighted fusion and Reciprocal Rank Fusion (RRF), so you do not have to manually merge and re-score results at the application layer.
- **Scalar-vector hybrid search.** This is where many vector databases fall down in practice. You do not just want "find me similar documents" — you want "find me similar documents written after January 2026 in the engineering department." zvec pushes scalar filters down into the vector index execution layer, avoiding full scans in high-dimensional space. Scalar fields can optionally build inverted indexes to further accelerate equality and range filtering.

zvec RAG feature Set covers the following

- Full CRUD: insert, update, delete documents in real time
- Schema evolution: adjust fields and index strategies over time
- Dense + sparse vectors: native multi-vector queries in a single call
- Built-in reranker: weighted fusion and RRF out of the box
- Hybrid search: scalar filters pushed into the vector index execution path
- Inverted indexes on scalar fields: accelerate equality/range filtering
- Built-in embedding and reranking extensions via the ecosystem

Resource Governance: The Feature Nobody Mentions

This is, to me, the most underappreciated aspect of zvec. And it is the one that matters most for edge and on-device deployments.

HNSW indexes are memory-hungry. During build or query, they can temporarily consume several times the raw data size.

On a cloud VM with 64 GB of RAM, who cares. On a mobile device, a desktop app, or a CLI tool, your application gets killed by the OOM Killer or triggers an Android ANR (Application Not Responding) dialog.

zvec provides three layers of memory management:

1. Streaming, chunked writes. Writes are processed in 64 MB chunks by default. You never hold the entire dataset in memory during ingestion.

2. On-demand loading via mmap. Enable `enable_mmap=true` and vector/index data is paged into physical memory on demand by the OS. Your collection can be larger than available RAM. This is important for edge devices where you might have a 2 GB vector collection but only 4 GB of total RAM.

3. Hard memory limiting (experimental). When mmap is not enabled, zvec maintains an isolated, process-level memory pool. You can explicitly cap its budget via `memory_limit_mb`.

On the concurrency side:

- **Index build threads** are configurable per-operation via a `concurrency` parameter, plus a global `optimize_threads` cap.
- **Query threads** are capped via a global `query_threads` setting.

This prevents the classic problem in GUI applications where background vector computation spawns too many threads, saturates the CPU, and causes UI stutter.

Here are also the edge deployment failure modes zvec addresses:

Edge Deployment Failure Modes zvec Addresses

OOM kill on mobile/embedded device HIGH

Trigger: HNSW index build consuming multiple times raw data size in memory

Detection: Process killed by OS OOM Killer or Android ANR

Mitigation: 64 MB streaming writes, mmap mode, hard memory_limit_mb cap

UI stutter in desktop/mobile apps MEDIUM

Trigger: Unconstrained thread spawning during vector index build or query

Detection: Main thread starvation, frame drops, ANR dialogs

Mitigation: Configurable optimize_threads and query_threads caps

Data loss after crash or forced exit HIGH

Trigger: Application killed mid-write (phone locked, CLI interrupted, power loss)

Detection: Corrupted or missing index on restart

Mitigation: Persistent storage with automatic crash recovery, thread-safe access

Real-World Patterns

zvec is still early (v0.2.0 as of February 2026), but interesting usage patterns are already emerging from the developer community.

Pattern 1: Agent memory and audit trail

zvec can serve as the **shared execution fabric and audit trail across agents.** Every message and tool call is traced, stored locally in zvec, and then recursively reviewed to improve future agent capabilities.

The architecture pairs QMD for per-agent knowledge retrieval with zvec for tracking what actually works.

Combined with Qwen 3.5, the whole stack runs locally on approximately 32 GB of RAM.

This is a pattern worth paying attention to: using an embedded vector database not just for retrieval but as an execution log that agents can search semantically.

Pattern 2: Privacy-preserving healthcare search

The healthcare benchmark I shared earlier is a deployment pattern. Semantic patient search where patient data never leaves the machine.

Describe a clinical presentation in natural language, find the most similar cases from 10,000 patient records. No keywords, meaning-based matching. The stack: zvec + fastembed (ONNX) + Polars, fully offline, `uv run`.

For regulated industries (healthcare, finance, legal), the "data never leaves the machine" property is a compliance requirement.

An in-process vector database that persists locally is architecturally incapable of leaking data over the network, because there is no network.

Pattern 3: The "zero-infrastructure RAG" for developer tools

The target scenario is a local RAG assistant on PC or mobile where users query local codebases, technical documents, or meeting notes via natural language, even without a network connection.

This is the IDE plugin use case, the local documentation search use case, the "I have 50 PDF papers and I want semantic search over them" use case.

The Tradeoffs You Need to Understand

I would not be doing my job if I only talked about what zvec does well. Here are the constraints and limitations you should factor into your decision:

- **Single-node only.** zvec is an embedded library. It runs on one machine. If you need distributed vector search across a cluster, this is not your tool. Milvus, Qdrant, or Weaviate are designed for that. zvec is for the (very large) class of applications where single-node is sufficient.
- **No GPU acceleration.** Unlike FAISS, which has GPU implementations for certain index types, zvec is CPU-only. Its performance comes from SIMD, cache-friendly layouts, and multi-threading but if you have spare GPU cycles and need to burn them on vector search, FAISS still owns that niche.
- **Platform support is still limited.** Linux x86_64, Linux ARM64, and macOS ARM64 only. No Windows support yet. No Python 3.13. If your deployment target is Windows desktops, you are waiting.
- **Python-only SDK.** The core is C++ (81.3% of the codebase), but the only user-facing SDK today is Python. Rust and other language bindings are on the roadmap but not shipped. If you are building a mobile app in Swift or Kotlin, you cannot use zvec directly yet though the C++ core theoretically supports it.
- **HNSW overhead at small scale.** As the independent benchmark showed, at 10K vectors NumPy brute-force is faster. HNSW's graph-based approach has overhead that only pays off at larger

scales. If your dataset is under 50K vectors and you do not need persistence or filtering, a simpler approach may suffice.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD is a spec-driven development workflow + prompt/agent harness that tries to prevent context rot by externalizing...

medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for...

medium.com

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Whether you think it's hype or not, people are already trying to run fully autonomous companies on OpenClaw.

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com
#qdrant#weaviate#ai-agent#vector-database#agentic-rag

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel